Class Loading and the Delegation Model

9 min read Updated 28 Jul 2026 Topic 44 of 65

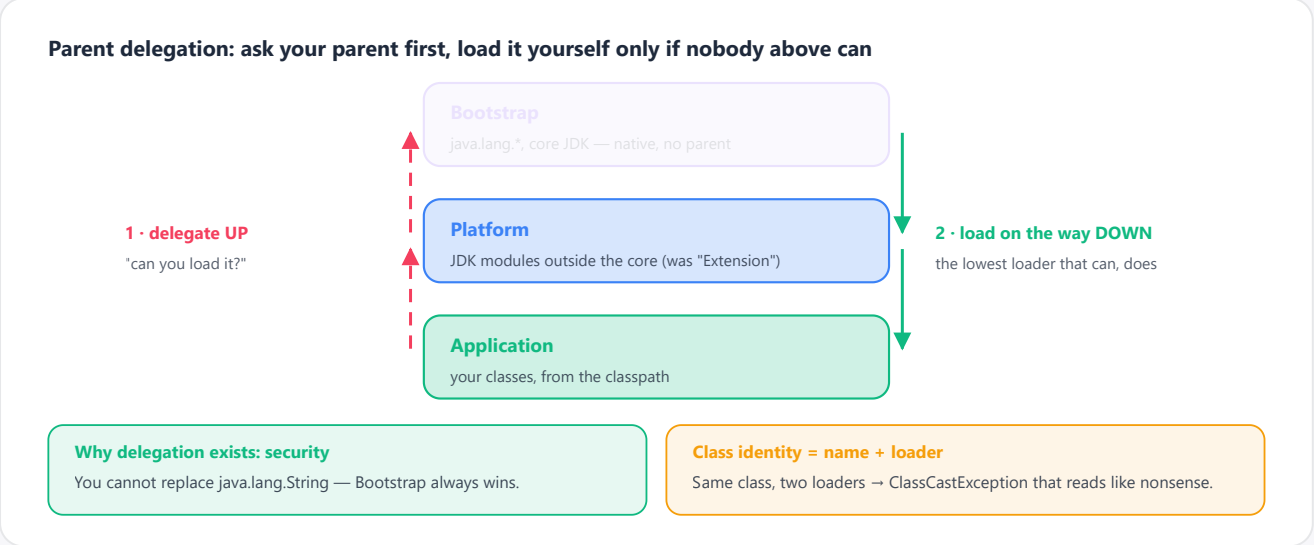
Bookmark Mark done

Why it matters

Class loading is invisible until it breaks, and then it produces the most confusing errors in Java: `ClassNotFoundException` versus `NoClassDefFoundError`, and the notorious "class X cannot be cast to class X". Every application server, plugin system and hot-reload mechanism is built on custom class loaders, so this is also where "how does Tomcat isolate two applications?" is answered.

The concept, simply

Classes are loaded **lazily** — on first active use, not at startup — by a hierarchy of loaders that follow **parent delegation**: before loading a class itself, a loader asks its parent, which asks its parent, all the way to the top.



Requests travel up, loading happens on the way back down. The highest loader that can load a class wins — which is what makes it impossible to substitute your own java.lang.String.

references), and **initialisation** (run static initialisers — Chapter 2's order).

How it works

The three built-in loaders (names changed in Java 9 with modules): **Bootstrap** loads the core JDK and is native code with no parent — `String.class.getClassLoader()` returns `null`, which is not a bug. **Platform** (formerly Extension) loads JDK modules outside the core. **Application** loads your classpath.

Why delegation exists is security. Without it, an attacker could put a malicious `java.lang.String` on the classpath and have it loaded instead of the real one. Delegation guarantees the Bootstrap loader always wins for core classes.

Class identity is name plus loader. Two loaders loading the same bytes produce two distinct types that are not assignable to each other. That is the source of the "`com.app.User` cannot be cast to `com.app.User`" error — the message looks absurd and is literally true.

Code examples

The two errors that look similar and mean different things:

JAVA

Copy

```java
// ClassNotFoundException — CHECKED. Something asked for a class BY NAME a
// the loader could not find it. Usually a missing dependency or a typo.
Class.forName("com.app.Missing");        // → ClassNotFoundException

// NoClassDefFoundError — an ERROR. The class was present at COMPILE time
// is missing (or failed to initialise) at run time. Two very different ca
//   1. The jar is not on the runtime classpath.
//   2. ⚠ The class FAILED ITS STATIC INITIALISER earlier. The first atten
//      throws ExceptionInInitializerError; every attempt after that throw
//      NoClassDefFoundError with no hint of the original cause.
// Always look for an earlier ExceptionInInitializerError in the log —
// that is where the real failure is.
```

Lazy initialisation, and what does and does not trigger it:

JAVA

Copy

```
class Config {
    static final int MAX = 100;                    // compile-time constan
    static final String HOST = System.getenv("H"); // computed at init
    static { System.out.println("Config initialised"); }
}

System.out.println(Config.MAX);    // does NOT initialise — the value was
                                   // INLINED into the caller at compile t
System.out.println(Config.HOST);   // DOES initialise → prints the message
new Config();                      // initialises
Config.class;                      // does NOT — a class literal is not ac
```

A custom loader, which is what plugin and app-server isolation really is:

JAVA

Copy

```
class PluginClassLoader extends ClassLoader {
    private final Path jar;
    PluginClassLoader(Path jar, ClassLoader parent) { super(parent); this.

    @Override
    protected Class<?> findClass(String name) throws ClassNotFoundExceptio
        byte[] bytes = readClassBytes(jar, name);   // ← override findClas
        return defineClass(name, bytes, 0, bytes.length);   // NOT loadCla
    }                                  // so delegation still runs first
}

// Two plugins, two loaders → two isolated copies of the same library.
// This is how Tomcat runs two webapps with conflicting dependency version
// and how OSGi and hot-reload work.
// ⚠ Tomcat deliberately INVERTS delegation for webapp classes (child-firs
//   so an app's bundled library wins over the container's — a documented
//   exception to the rule, not a violation of it.
```

The class-identity trap:

```
Class<?> a = loaderA.loadClass("com.app.User");
Class<?> b = loaderB.loadClass("com.app.User");
System.out.println(a == b);           // false — different types!
Object u = a.getDeclaredConstructor().newInstance();
// com.app.User user = (com.app.User) u;
// → ClassCastException: class com.app.User cannot be cast to class com.ap
//   The message names the loaders in brackets — that is the clue.
```

Under the hood

Verification is a real security boundary. Before executing a loaded class the JVM proves properties about its bytecode: the operand stack never underflows, locals are read with the type they were written with, control flow never leaves the method. This is what historically made running untrusted applets possible, and it is why a corrupt jar fails loudly at load rather than corrupting the heap.

Class initialisation is thread-safe and exactly-once, guaranteed by the JVM under an internal lock — the property that makes the holder idiom (Chapter 2) a correct lazy singleton. The same lock means a static initialiser that blocks waiting on another class which is itself waiting can deadlock, with stack traces that look idle.

Class unloading barely happens. A class can only be collected when its *loader* becomes unreachable, so ordinary application classes live for the life of the JVM. This is why repeated hot redeploys leak Metaspace: each deploy creates a new loader, and if anything — a `ThreadLocal`, a JDBC driver registration, a running thread — still references the old one, none of its classes can be freed. That is the classic `OutOfMemoryError: Metaspace` in an app server.

Java 9 modules added a second layer on top: readability between modules and strong encapsulation of internals. Since Java 16 that encapsulation is enforced by default, which is why libraries doing deep reflection into JDK internals broke on newer releases.

Common mistakes

- **Confusing the two errors.** `ClassNotFoundException` = asked by name, not found. `NoClassDefFoundError` = was there at compile time, missing or failed to initialise now.
- **Missing the earlier `ExceptionInInitializerError`.** The real cause is usually further up the log.
- **Overriding `loadClass` instead of `findClass`.** Breaks delegation and the security model.
- **Expecting a class to load at startup.** Loading is lazy — errors appear on first use, often mid-request.
- **Assuming a constant read triggers initialisation.** Compile-time constants are inlined.
- **Ignoring loader leaks on redeploy.** One lingering reference pins every class in that loader.
- **Heavy work in static initialisers.** Slow first use, and a possible class-init deadlock.

Interview lens

Q: "Explain the class loading process."

A: Three phases. Loading finds the bytes and creates the `Class` object. Linking verifies the bytecode, prepares static fields with defaults, and resolves symbolic references. Initialisation runs static initialisers and static field assignments in source order. It is lazy — triggered by first active use such as instantiation, a static method call, or reading a non-constant static field.

Q: "What is parent delegation and why does it exist?"

A: A loader asks its parent to load a class before trying itself, so requests travel up to Bootstrap and loading happens on the way back down. It exists for security and consistency: you cannot substitute your own `java.lang.String`, because Bootstrap always gets asked first, and every loader in the chain sees the same core classes.

A: `ClassNotFoundException` is a checked exception from asking for a class by name — `Class.forName` or `loadClass` — and not finding it. `NoClassDefFoundError` is an `Error`: the class was present at compile time but is missing at run time, or, more subtly, it failed its static initialiser earlier. In that second case the first attempt threw `ExceptionInInitializerError` and every subsequent one throws `NoClassDefFoundError` with no cause — so I look further up the log for the original failure.

Q: "Why would `User cannot be cast to User` happen?"

A: Because class identity is the fully-qualified name *plus* the loader. Two loaders loading the same bytes produce two distinct, incompatible types. It usually means the same jar is on both the container's and the application's classpath, or an object crossed a plugin boundary. The exception message names the loaders in brackets, which is how you confirm it.

Q: "Why do redeploys leak memory?"

A: Classes can only be unloaded when their loader is unreachable. Each redeploy creates a new loader, and if anything still references the old one — a `ThreadLocal` on a pooled container thread, a registered JDBC driver, a running thread, a JVM-level cache — that loader and every class it defined stay alive. Repeat a few times and you get `OutOfMemoryError: Metaspace`. Diagnosing it means finding what still references the old loader, with a heap dump.

Q: "Would you ever write a custom class loader?"

A: For plugin isolation, hot reload, or loading classes from a non-standard source such as a database or an encrypted archive. The rule is to override `findClass` rather than `loadClass`, so the delegation and security model stay intact. Tomcat is the well-known exception — it deliberately inverts delegation for webapp classes so an application's bundled library beats the container's.

- Phases: **load → link (verify, prepare, resolve) → initialise**. Lazy, on first active use.
- Loaders: **Bootstrap → Platform → Application**. Bootstrap is native; `getClassLoader()` returns `null`.
- **Parent delegation**: ask up, load down. Exists for security.
- **Class identity = name + loader** → "X cannot be cast to X".
- `ClassNotFoundException` = by-name lookup failed (checked). `NoClassDefFoundError` = missing at run time **or a failed static init**.
- Look for an earlier `ExceptionInInitializerError` — that is the real cause.
- Reading a **compile-time constant does not initialise** the class.
- Class init is **thread-safe and once** → holder idiom; can deadlock if it blocks.
- Unloading needs an unreachable **loader** → redeploy leaks → Metaspace OOM.
- Custom loader: override `findClass`, not `loadClass`.

QUICK CHECK — no marks, just for you.

1. What is parent delegation and why does it exist?

- ◯ Loaders share one flat namespace and the fastest wins
- ◯ The JVM picks a loader at random for each class
- ◯ A loader asks its parent to load a class before trying itself — so you cannot substitute your own java.lang.String, because Bootstrap is always asked first
- ◯ A loader delegates to its children first, so application classes win

Check answer

2. You see 'class com.app.User cannot be cast to class com.app.User'. What causes it?

- ◯ A generics erasure problem

- Two versions of the class with different fields
- Class identity is the fully-qualified name PLUS the loader, so two loaders produce two distinct, incompatible types

Check answer

3. What is the difference between ClassNotFoundException and NoClassDefFoundError?

- ClassNotFoundException means a by-name lookup failed; NoClassDefFoundError means the class was present at compile time but is missing now — OR it failed its static initialiser earlier
- NoClassDefFoundError only occurs with reflection
- ClassNotFoundException is thrown by the JVM and NoClassDefFoundError by the compiler
- They are the same; one is just newer

Check answer

‹ Previous
Atomics, CAS and Lock-Free Concurrency

Next ›
JVM Memory Areas

LEARN MORE

ARTICLES & DOCS

ClassLoader — API docs
docs.oracle.com

VIDEOS

"classloading" — video explainers
YouTube

classloading interview questions
YouTube

Hand-picked references for this lesson.